

Tarea 2 — cmatch: threads y sincronización

Sistemas Operativos

Felipe Bustos López

felipe.bustos@mail.udp.cl

Cristóbal Sepúlveda Maldonado

cristobal.sepulveda@mail.udp.cl

Profesor: (*indicar*)

Mayo de 2026

1. Introducción

En esta tarea se construyó **cmatch**, un motor de emparejamiento concurrente para torneos de *tic-tac-toe* (gato). El sistema gestiona N jugadores que buscan activamente formar partidas uno contra uno, compitiendo por un número limitado de K tableros. Cada jugador y cada tablero activo se modela como un *thread* independiente, por lo que el problema central no es la lógica del juego — que es trivial — sino la coordinación segura de muchos hilos que comparten recursos: la cola de espera, el puntaje ELO de los jugadores y los tableros disponibles.

El objetivo de este informe es describir el diseño del programa, justificar los mecanismos de sincronización empleados (únicamente la librería POSIX Threads, **pthread**, según exige el enunciado) y comentar los mayores desafíos que debimos resolver durante la implementación.

2. Diseño del programa

El programa se organiza en torno a dos tipos de hilos. Los **hilos de jugador** ejecutan un ciclo de vida: ingresan a la cola de espera, buscan un oponente válido, juegan una partida y, al terminar, deciden autónomamente — según la probabilidad **REENTER_PROBABILITY** — si reingresan a la cola o finalizan su ejecución. Los **hilos de tablero** se crean por cada partida y actúan como árbitros: coordinan los turnos, validan el estado del juego y, al finalizar, actualizan el ELO y liberan el tablero.

El **emparejamiento** sigue las dos reglas pedidas. Un jugador en espera busca entre los demás jugadores encolados aquel cuya diferencia de ELO no supere Δ_{max} (**MAX_ELO_DIFF**); si hay varios candidatos válidos, se elige al que lleva más tiempo esperando, lo que se resuelve asignando a cada jugador un número de llegada creciente y prefiriendo el menor. El jugador que encuentra rival y consigue un tablero libre actúa como “anfitrión”: reclama a ambos, crea la partida y lanza el hilo del tablero; el otro queda como “invitado”. Así, ningún momento hay más de K partidas simultáneas.

Al terminar una partida, el ELO de ambos jugadores se actualiza con el sistema estándar de Elo,

$$E_A = \frac{1}{1 + 10^{(ELO_B - ELO_A)/400}}, \quad ELO_{nuevo} = ELO_{ant} + K_{Elo}(S - E),$$

donde $S \in \{1, 0, 5, 0\}$ según el jugador gane, empate o pierda. La configuración completa (cantidad de jugadores, tableros, factor K_{Elo} , retardo entre jugadas, etc.) se lee de un archivo con formato **.env**. Finalmente, el programa atiende la señal **SIGINT** para realizar un cierre ordenado y guarda en disco un *snapshot* con el estado de los jugadores, que permite reanudar la simulación tal como quedó.

3. Mecanismos de sincronización utilizados

Decidimos usar **exclusivamente mutex y variables de condición** de `pthread`, evitando incluso los semáforos POSIX (`<semaphore.h>`), ya que el enunciado restringe la sincronización a la librería de hilos. Las estructuras empleadas y su justificación son:

- **Un mutex global (`g_mutex`)** que protege la “tabla de jugadores”: la cola de espera, el estado de cada jugador, su ELO, sus estadísticas y el contador de tableros libres. Optamos por un *bloqueo de grano grueso* porque estos datos se consultan y modifican de forma entrelazada durante el matchmaking; un único candado garantiza que, por ejemplo, jamás se lea un ELO mientras se está actualizando, eliminando las condiciones de carrera sobre la sección crítica del puntaje.
- **Una variable de condición (`g_cv`)** sobre la que los jugadores esperan cuando no encuentran rival o no hay tablero libre. Esto evita el *busy-waiting*: el hilo se bloquea y solo es despertado cuando llega un candidato nuevo a la cola o cuando se libera un tablero.
- **Un contador de tableros libres** (en vez de un semáforo) protegido por `g_mutex`. Se decrementa al iniciar una partida y se incrementa al terminarla, lo que impone el tope de K partidas simultáneas usando solo primitivas de `pthread`.
- **Un mutex y una variable de condición propios de cada partida (`m->mtx`, `m->cv`)**, que sincronizan al hilo del tablero con los dos jugadores por turnos. El tablero avisa a quién le toca, espera su jugada y cambia el turno; los jugadores esperan bloqueados su turno. Como tres hilos comparten la misma condición, usamos `pthread_cond_broadcast` y re-evaluamos la condición tras cada despertar para evitar despertares perdidos. Tener un candado por partida permite que las distintas partidas avancen en paralelo sin contención entre sí.
- **Una bandera volatile `sig_atomic_t` para SIGINT**: el manejador solo marca la bandera (única operación segura en ese contexto) y es el hilo principal quien, al detectarla, toma `g_mutex`, deja de aceptar nuevas partidas y despierta con `broadcast` a todos los jugadores en espera para que terminen ordenadamente.

4. Principales desafíos

El mayor desafío fue **coordinar tres hilos en una misma partida** (dos jugadores y el tablero) por turnos, sin caer en despertares perdidos ni en interbloqueos. Lo resolvimos con un esquema en el que el tablero actúa como único árbitro, todas las transiciones de estado se anuncian con `broadcast` y cada hilo vuelve a comprobar la condición al despertar, de modo que el orden de ejecución de los hilos no afecta la corrección.

Un segundo desafío fue el **manejo del ciclo de vida de la memoria de cada partida**: liberar la estructura compartida sin que algún hilo la siga usando. Para ello la partida lleva un contador de jugadores que ya salieron de ella; el hilo del tablero espera a que ambos jugadores hayan terminado antes de liberar la memoria, garantizando que nadie acceda a una región ya liberada.

También fue importante lograr un **cierre realmente ordenado**: tras recibir SIGINT no deben quedar hilos colgados. Para esto el hilo principal espera con `pthread_join` a todos los jugadores y, además, a que el contador de tableros activos llegue a cero antes de guardar el *snapshot* y terminar. Por último, respetar el tope de K partidas *sin busy-waiting* nos obligó a combinar el contador de tableros con la variable de condición, en lugar de recurrir a esperas activas.

5. Conclusiones

Durante el desarrollo de esta tarea, como grupo decidimos implementar `cmatch` en C (C17), ya que es un lenguaje con el que nos sentimos cómodos y que expone directamente las primitivas de `pthread`, lo que nos permitió razonar con claridad sobre cada sección crítica. La principal decisión de diseño fue separar la sincronización en dos niveles: un candado global para el estado compartido de los jugadores y candados independientes por partida. Esta separación mantiene la corrección del recurso más delicado — el ELO — y, al mismo tiempo, permite que las partidas se jueguen en paralelo sin estorbarse.

Somos conscientes de que el uso de un único mutex global para la tabla de jugadores es una solución de grano grueso que limita la concurrencia del matchmaking; la preferimos por sobre un esquema de candados más finos porque prioriza la corrección y es mucho más fácil de razonar, dejando la optimización como un trabajo futuro. La implementación se validó con la herramienta ThreadSanitizer, que no reportó condiciones de carrera, y se verificó que el cierre por `SIGINT` y la restauración desde el *snapshot* funcionan correctamente. En conjunto, la tarea nos permitió aplicar de forma concreta los conceptos de exclusión mutua, variables de condición y comunicación entre hilos vistos en el curso.

Anexo

Repositorio con el código fuente, el `Makefile`, la configuración de ejemplo y las instrucciones de uso: https://github.com/sephppx/Tarea_02-Sistema-Operativos.